Mikrocomputertechnik 1

Einheit 8: I/O & Systemarchitektur

Prof. Dr.-Ing. Daniel Klünder

Folie 1 / 84

I/O & Systemarchitektur

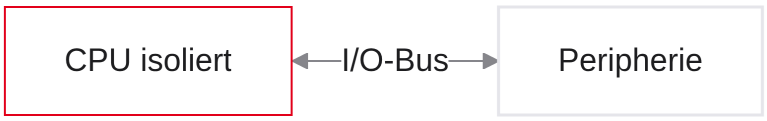
Notizen

Mikrocomputertechnik 1 verbindet das bekannte Rechenwerk mit Ein- und Ausgabe, kurz I/O für Input/Output. Heute geht es darum, Geräte über Adressen anzusprechen und Dienste einer Ausführungsumgebung zu nutzen. Vorausgesetzt sind Register, Loads und Stores sowie das vereinfachte Pipeline-Modell. Ein Betriebssystemkern wird nicht vorausgesetzt.
Kontrollfrage: Liegt der Schwerpunkt auf schnellerer Arithmetik? Nein, auf der Verbindung zur Umgebung. Die nächste Folie nennt die fehlende Fähigkeit.

Folie 2 / 84

Die Brücke zur Außenwelt

- Herzlich willkommen zur achten Vorlesung in Mikrocomputertechnik 1
- Letzte Einheiten: Datenpfad, Steuerwerk, Pipelining
- Unser 5-stufiger RISC-V-Prozessor ist schnell und logisch korrekt
- Letztes Problem: der Chip ist taub und stumm
- Heute: die Brücke zur Außenwelt (Input/Output)



Notizen

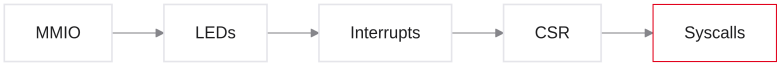
Datenpfad, Steuerwerk und Pipeline erklären, wie Befehle rechnen. Hoher Durchsatz beantwortet noch nicht, wie ein Schalter das Programm beeinflusst oder eine LED, eine Leuchtdiode, ein Ergebnis zeigt. Das bisherige Modell ist ein Lehrchip, nicht ein vollständiges System. CPU bedeutet Central Processing Unit. RISC-V ist die Befehlssatzarchitektur, kurz ISA für Instruction Set Architecture.

Kontrollfrage: Bedeutet logisch korrekt schon ein fertiges Gerät? Nein. Die Agenda zerlegt die Brücke in Teilfragen.

Folie 3 / 84

Agenda für den heutigen Tag

1. Memory-Mapped I/O — wie spricht die CPU mit Peripherie?
2. Schalter & LEDs — Ansteuerung und Entprellung
3. Polling vs. Interrupts — nicht sinnlos warten
4. Das CSR-System — versteckte Register für Systemzustände
5. Syscalls & OS — Hilfe vom Betriebssystem (`ecall`)
6. Labor (Ripes/Venus) — virtuelle LEDs und Konsolenausgaben



Notizen

Sechs Fragen: Wo liegt das Gerät? Wie lesen wir seine Bits? Wie warten wir? Welchen Zustand braucht eine Unterbrechung? Wie fordert Software einen Dienst an? Wie prüfen wir das im Labor? MMIO bedeutet Memory-Mapped I/O, Gerätezugriff über Adressen. CSR bedeutet Control and Status Register. OS bedeutet Operating System. ecall ist der Environment Call. Ripes zeigt Geräte, Venus die Konsole. Das sind Kursaufgaben, keine Ausschlüsse.
Kontrollfrage: Sind Ripes und Venus dasselbe Modell? Nein. Zuerst der bekannte Prozessor.

Folie 4 / 84

Rückblick: Der isolierte High-Speed-Chip

- Status aus Einheit 7: Pipeline mit hohem Durchsatz
- ALU, PC und RAM arbeiten fehlerfrei
- Alle Daten bleiben auf dem Die gefangen
- Kein Knopfdruck kann das laufende Programm beeinflussen

```
Ein Prozessor ohne I/O ist wie ein Taschenrechner
ohne Display und ohne Tasten: physikalisch nutzlos.
```

Notizen

Die ALU, Arithmetic Logic Unit, rechnet. Der PC, Program Counter, hält die Befehlsadresse. RAM, Random Access Memory, ist der Arbeitsspeicher im Modell. Ein Ergebnis in s0 bleibt unsichtbar, solange kein Ausgabegerät angeschlossen ist. Daten müssen nicht ausschließlich auf einem Die liegen. Die Aussage gilt für das bisherige Modell ohne Eingabe, nicht für jeden realen Chip.
Kontrollfrage: Beeinflusst ein Knopf das bisherige Programm? In diesem Modell nein. Die Metapher macht

die Grenze anschaulich.

Folie 5 / 84

Das Brain-in-a-Jar-Problem

- Gedankenexperiment: Gehirn im Einmachglas
- Rechnet und denkt — ohne Sinnesorgane
- Unsere CPU ist genau dieses Gehirn
- I/O ist das Nervensystem, das wir anschließen
- Reize empfangen, Aktionen in der Welt auslösen

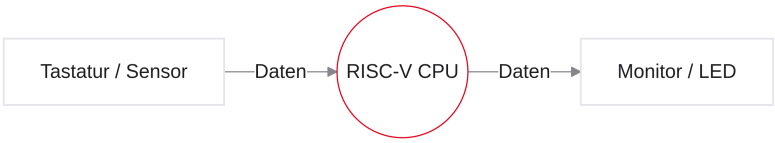
Notizen

Ein Sensor liefert ein elektrisches Signal, ein Aktor eine Wirkung. Das Interface übersetzt den Zustand in ein lesbares Register. Die CPU denkt nicht und besitzt keine zusätzlichen Sinnesorgane. Die Metapher motiviert nur diese fehlende Verbindung.
Kontrollfrage: Liefert ein Schalter eine fertige Dezimalzahl? Nein, ein Signal, das ein Register abbildet. Als Nächstes heißt das Peripherie.

Folie 6 / 84

Peripherie (Die Außenwelt)

- Alles außer der reinen Recheneinheit: Peripherie
- Input (Sensoren): Tastatur, Maus, Temperatur, Schalter, Netz (Empfang)
- Output (Aktoren): Monitor, LED, Lautsprecher, Motor, Netz (Senden)
- Frage: Wie anbinden, ohne für jedes Gerät ein neues Steuerwerk?



Notizen

Peripherie meint hier Geräte für Kommunikation, Messung oder Steuerung. Speicher und interne Prozessorstrukturen sind nicht automatisch Peripherie. Ein Sensor misst, ein Aktor wirkt, ein Netzinterface kann beides. Ein Motor wird über Treiber und Leistungselektronik angesteuert, nicht direkt durch einen Registerwert an den Wicklungen.
Kontrollfrage: Ist RAM Peripherie im selben Sinn wie eine LED? Nein. Eine gemeinsame Zugriffsmethode führt zu MMIO.

Folie 7 / 84

Memory-Mapped I/O

Notizen

Memory-Mapped I/O ordnet Geräteregister Adressen zu. Der Prozessor nutzt passende Speicherbefehle. Die Leitfrage lautet, woran das System RAM und Gerät unterscheidet. Decoder und Zugriff folgen. Kontrollfrage: Braucht MMIO einen eigenen Portbefehl? In diesem Modell nein. Zuerst die beiden Zugriffsarten.

Folie 8 / 84

Wie spricht die CPU mit Peripherie?

- Zwei historische Philosophien:
- Port-Mapped I/O (Komplexität)
- Memory-Mapped I/O (Einfachheit)
- Intel x86: historisch oft Port-Mapped
- ARM und RISC-V: konsequent Memory-Mapped I/O

Notizen

Port-Mapped I/O nutzt einen eigenen logischen Port-Adressraum und eigene Befehle. Memory-Mapped I/O nutzt Adressen wie Speicher. x86 kann beides. RISC-V-Geräte liegen auf plattformspezifischen Adressen. Die ISA allein legt weder Gerät noch Adresse fest.
Kontrollfrage: Ist MMIO die einzige mögliche Einteilung? Nein. Das x86-Beispiel zeigt den Port-Raum.

Folie 9 / 84

Port-Mapped I/O (x86)

- Separater I/O-Bus und eigene ISA-Befehle: `IN`, `OUT`
- Beispiel: `IN AL, 0x60` (klassischer Tastatur-Port)
- Nachteile: komplexeres Steuerwerk, begrenzter Port-Raum, mehr Silizium

```
# Alter x86-Stil (Port-Mapped I/O):
MOV DX, 0x03F8     # COM1
OUT DX, AL         # Byte aus AL an den Port
```

Notizen

IN und OUT adressieren einen logischen Port, nicht zwingend einen eigenen physischen Bus. DX kann die Portnummer halten, AL das Byte. COM1 und der Tastaturport 0x60 sind verschiedene historische Zuordnungen. Ein normales lw trifft diesen Port-Raum nicht. Siliziumkosten ohne Beleg bleiben weg.
Kontrollfrage: Erreicht lw automatisch Port 0x60? Nein. MMIO nutzt den adressierten Pfad.

Folie 10 / 84

Memory-Mapped I/O (MMIO)

- RISC-V-Weg: kein neuer I/O-Bus, keine neuen Befehle
- Philosophie: Hardware tarnen als Arbeitsspeicher
- Kommunikation wie mit einer Array-Zelle im RAM
- Bekannte Befehle: `lw` und `sw`

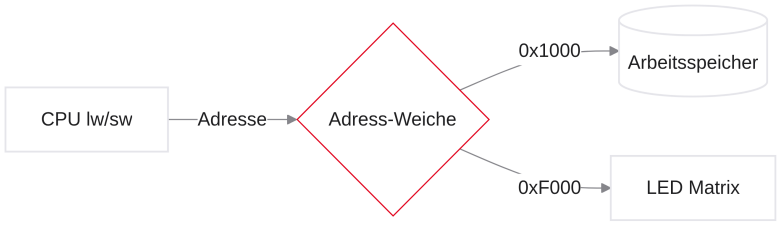
Notizen

Gleiche Adressierung bedeutet nicht gleiche Zelle. Ein Geräteregister ist kein frei nutzbarer RAM. Für die hier gewählten 32-Bit-Register passen lw und sw. Andere Geräte können andere Breiten und Nebenwirkungen haben. Keine eigenen Portbefehle heißt nicht, dass keine Bridges existieren.
Kontrollfrage: Darf man ein Geräteregister wie ein Arrayelement cachen? Nicht ohne Vertrag. Die Adresse wählt das Ziel.

Folie 11 / 84

Die Illusion des Speichers

- CPU legt 32-Bit-Adresse auf den Bus — glaubt, sie spricht mit RAM
- Adress-Decoder als Weiche
- Spezielle Adressen (z. B. `0xF0000000`) schalten RAM ab und Peripherie an
- Gerät legt Daten auf den Bus; CPU lädt ahnungslos nach `t0`



Notizen

Schritte: Adresse bilden, Zugriff übertragen, dekodieren, Ziel wählen, Daten liefern. Der Prozessor wird nicht betrogen. 0xF0000000 ist hier illustrativ, solange der Export sie nicht bestätigt. RV32 bedeutet nicht 32 Adresspins an jedem Chip.
Kontrollfrage: Ist jede hohe Adresse ein Gerät? Nein, das legt die Karte fest. Der Decoder braucht diese

Karte.

Folie 12 / 84

Der Adressraum (Address Map)

- 32-Bit-Adressraum: $2^{32} = 4\,294\,967\,296$ Bytes (4 GB)
- Feste Address Map im Chip-Handbuch
- Beispielhafte Aufteilung:
- `0x00000000`–`0x0FFFFFFF`: Boot-ROM / Code
- `0x10000000`–`0x7FFFFFFF`: RAM (Heap, Stack, `.data`)
- `0xF0000000`–`0xFFFFFFFF`: MMIO (Sensoren, LEDs)

```
lw aus 0x10000000 -> Array-Element im RAM
lw aus 0xF0000000 -> z.B. Pixel/Sensor-Wert
```

Notizen

2 hoch 32 Byte sind 4 GiB, nicht automatisch 4 dezimale GB und nicht der eingebaute RAM. Die gezeigte Aufteilung ist ein Lehrbeispiel. Der Bereich zwischen RAM und MMIO kann reserviert sein. RAM liegt nicht allgemein unten und Geräte nicht allgemein oben.
Kontrollfrage: Sind 4 GiB eingebauter Speicher? Nein. Die Karte wählt einen Baustein.

Folie 13 / 84

Der Adress-Decoder (Hardware)

- Oft Teil des SoC / der Bridge-Logik
- Jedes Bauteil: Chip-Select (`CS`)
- `CS = 0`: Bauteil schläft; `CS = 1`: aktiv
- Decoder: AND-Netzwerk auf oberen Adressbits (z. B. `0xF...`)

Notizen

SoC bedeutet System on Chip. Ein Chip Select, kurz CS, lässt den gewählten Baustein am Zugriff teilnehmen. In diesem vereinfachten Bild ist CS aktiv-high. Andere Interfaces sind aktiv-low. Obere Adressbits sind ein Lehrdecoder, keine universelle Schaltung. Zwei gleichzeitig antwortende Ziele sind zu vermeiden. Unzugeordnete Adressen sind eine Plattformfrage.
Kontrollfrage: Weckt CS den Baustein aus dem Schlaf? Es wählt ihn für den Zugriff. Danach gelten Load und Store.

Folie 14 / 84

Load/Store für I/O

- MMIO macht Bare-Metal-Assembler einfach
- Schalter lesen = `lw`; Motor starten = `sw` mit 1

```
li s1, 0xF0000000    # Thermometer-Basis
li s2, 0xF0000010    # Warn-LED

lw t0, 0(s1)         # MMIO-Read
li t1, 100
blt t0, t1, Safe

li t2, 1
sw t2, 0(s2)         # MMIO-Write: Alarm-LED
Safe:
```

Notizen

Die Adressen im Thermometerbeispiel sind fiktiv. Der Wert 100 ist ohne Einheit keine Temperatur. lw liest das Register, der Vergleich entscheidet, sw schreibt die Warn-LED. Ein Pfad, der den Alarm nur setzt, löscht ihn nicht. Für eine Schwellwertanzeige muss auch der inaktive Zustand geschrieben werden. Kontrollfrage: Schaltet der bisherige Safe-Pfad einen Alarm aus? Nicht, wenn nur der aktive Zweig schreibt. Ripes-Geräte haben eigene Verträge.

Folie 15 / 84

Praxis der MMIO (Schalter und LEDs)

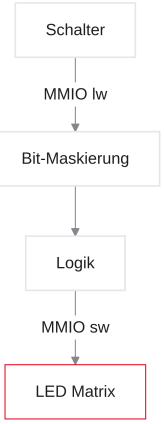
Notizen

Ziel ist lauffähiger I/O-Code aus einer Registerkarte. Switches, D-Pad und LED-Matrix sind verschiedene Modelle. Eine Bitmaske isoliert Eingabebits. Ein RGB-Wort, Rot Grün Blau, ist nicht dieselbe Bitmaske. Kontrollfrage: Darf man das Schalterwort unverändert in ein Farbregister schreiben? Für die Matrix hier nein. Zuerst die Formate.

Folie 16 / 84

Von der Theorie zur Laborpraxis

- Input: D-Pad / Schalter
- Einzelne Bits maskieren
- Output: LED-Matrix
- Gefahr: volatile Speicher (Compiler-Caches)



Notizen

Schritte: Gerät anlegen, Export lesen, Eingabe lesen, Bit isolieren, Farbe bilden, schreiben. volatile in C kennzeichnet extern veränderliche Zugriffe. Das ist keine Hardware-Cache-Abschaltung. Registerhaltung durch den Compiler ist eine andere Ebene als ein Daten-Cache.
Kontrollfrage: Ist volatile ein Cache-Attribut? Nein. Zuerst das gepackte Schalterregister.

Folie 17 / 84

Input: Switches (Bitmaske)

- Beispiel: Switches an `0xF0000000` (Ripes)
- `lw` liefert ein 32-Bit-Wort als **Bit-Map**
- Jedes Bit = ein physikalischer Schalter (1 = an)

Bits 31:4	Bit 3	Bit 2	Bit 1	Bit 0
0…	SW3	SW2	SW1	SW0

Notizen

Im konkreten Aufbau mit mindestens vier Schaltern ist SW0 Bit 0. SWITCHES_0_BASE kommt aus dem Export. 0b0101 bedeutet SW0 und SW2 aktiv. Die Anzahl und die elektrische Polarität hängen vom Gerät ab. Eins muss nicht gedrückt bedeuten.
Kontrollfrage: Ist 0b0101 die Farbe Rot? Nein, eine Eingabemaske. Das D-Pad trennt die Richtungen.

Folie 18 / 84

Input: D-Pad (eigene Register)

- In Ripes: **vier separate 32-Bit-Register**, je Richtung eines
- Offsets vom D-Pad-Basis: UP `+0`, DOWN `+4`, LEFT `+8`, RIGHT `+12`
- LSB = Tastenzustand (1 = gedrückt); restliche Bits typisch 0
- Ripes-Symbole: `D_PAD_0_UP`, `D_PAD_0_DOWN`, …

Notizen

Vier Richtungen können vier Register mit Offsets 0, 4, 8 und 12 sein. Das ist am Export zu prüfen, nicht als universelle Hardware. Der Tastenzustand wird maskiert, reservierte Bits nicht als null angenommen. Basis plus Offset ist die Registeradresse.
Kontrollfrage: Liegt oben im selben Wort wie die Schalter? In diesem Modell nein. Bei den Schaltern isoliert die Maske ein Bit.

Folie 19 / 84

Switches auslesen und maskieren

- Problem: mehrere Schalter gleichzeitig (z. B. `0b0101`)
- `beq` auf den Rohwert scheitert an Stör-Bits
- Lösung: Bit isolieren mit `andi` und Maske

```
li s1, 0xF0000000       # Switches-Basis
lw t0, 0(s1)

# Bit 2 (SW2): Maske 4 = 0b0100
andi t1, t0, 4
bne t1, zero, Jump
```

Notizen

0b0101 und 0b0100 ergeben 0b0100, also 4. Die Maske 4 trifft Bit 2, SW2, nicht den vierten Schalter. bne gegen zero erkennt Aktivität, obwohl das Ergebnis 4 und nicht 1 ist. Ein Vergleich des ganzen Wortes mit 4 scheitert, wenn zusätzlich SW0 aktiv ist. s1 hält die Basis, t0 den Rohwert, t1 das Maskenergebnis.
Kontrollfrage: Ist das Ergebnis nach andi immer 0 oder 1? Hier 0 oder 4. Das D-Pad wählt zuerst das Register.

Folie 20 / 84

D-Pad auslesen (eine Richtung)

```
li s1, 0xF0000000       # D-Pad-Basis
lw t0, 0(s1)            # UP    (+0)
lw t1, 4(s1)            # DOWN  (+4)
lw t2, 8(s1)            # LEFT  (+8)
lw t3, 12(s1)           # RIGHT (+12)
bne t0, zero, UpPressed
```

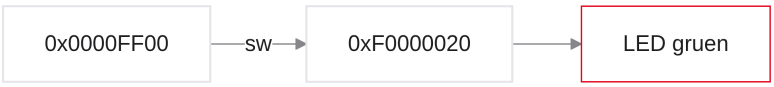
Notizen

Jede Richtung ist ein eigener lw. Wenn nur das niederwertige Bit, das LSB, den Zustand trägt, reicht ein Vergleich des ganzen Wortes mit ungleich null nicht. andi mit 1 isoliert es. Langes Halten ist ein Pegel. Ein Zähler braucht die Freigabe, sonst zählt er bei jeder Schleife erneut. Warten auf Loslassen ist noch keine mechanische Entprellung.
Kontrollfrage: Zählt ein gehaltener Pegel mehrfach? In einer naiven Schleife ja. Die Matrix braucht ein Farbwort.

Folie 21 / 84

Output: Die LED-Matrix

- Beispiel-Adresse `0xF0000020`
- `sw` brennt Bit-Muster auf die Dioden
- RGB oft als `0x00RRGGBB`
- `0x00FF0000` → knallrot



Notizen

Ein Pixel ist hier ein 32-Bit-Farbregister. 0x00RRGGBB bedeutet je ein Byte Rot, Grün und Blau. 0x00FF0000 ist Rot, 0x0000FF00 Grün. Ein Store schaltet die Anzeige im Modell, er brennt nichts dauerhaft ein. Die Adresse ist Basis plus 4 mal Index, wenn der Index gültig ist.
Kontrollfrage: Ist die Maske 8 schon Rot? Nein. Eingaben können sich ohne Store ändern.

Folie 22 / 84

Flüchtigkeit von I/O-Registern (Volatile)

- RAM: geschrieben = gelesen (passiv)
- MMIO: lebendig und flüchtig (volatile)
- Schalter-Register ändert sich durch externe Physik — ohne `sw` der CPU

```
RAM: Tresor – ändert sich nur, wenn ICH schreibe.
I/O: Fenster – die Welt draußen ändert sich ständig.
```

Notizen

Ein MMIO-Register kann zwischen zwei Lesungen einen anderen Wert liefern, weil das Gerät den Zustand ändert. Das ist keine Datenverlust-Flüchtigkeit ohne Strom. Auch RAM kann sich durch DMA, Direct Memory Access, oder andere Kerne ändern. Manche Zugriffe haben Nebenwirkungen. volatile ist der C-Qualifizierer für wiederholte Zugriffe, nicht die elektrische Eigenschaft selbst.
Kontrollfrage: Bleibt ein Tasterregister zwischen zwei lw garantiert gleich? Nein. Der Compiler darf die Wiederholung nicht wegoptimieren.

Folie 23 / 84

Caching-Gefahren bei I/O

- Zwei getrennte Gefahren:
  - Compiler: ohne `volatile` eliminiert `-O3` wiederholte Loads
  - Hardware-Cache: MMIO muss in der Memory Map non-cacheable sein
- Bug: Knopfdruck wird nie bemerkt
- In Assembler: jedes `lw` in der Schleife ist ein Bus-Zugriff

```c
/* gefährlich ohne volatile: */
int *button = (int *)0xF0000000;
while (*button == 0) { }

/* Rettung (Compiler): */
volatile int *button = (volatile int *)0xF0000000;
```

Notizen

Ohne volatile kann der Compiler eine wiederholte Abfrage in ein Register ziehen. Das ist nicht nur ein -O3-Thema. volatile ist keine Speicherbarriere und keine Thread-Sperre. Die Aussage, jedes lw sei ein Buszugriff, gilt hier nur für das uncached-MMIO-Lehrmodell. Cacheattribute und Speicherordnung sind weitere Ebenen.
Kontrollfrage: Schaltet volatile den Cache aus? Nein. Im Labor wird der Zugriff sichtbar.

Folie 24 / 84

I/O-Module in Ripes (Vorschau)

- Eigener I/O-Tab: virtuelle Boards einstecken
- D-Pad, Switches, LED-Matrizen
- Ripes emuliert den MMIO-Decoder
- Mausklick auf Schalter → Bit im gespiegelten Adressraum

Notizen

Der I/O-Tab modelliert Register und sichtbare Wirkungen. Er simuliert nicht automatisch mechanisches Prellen oder eine Interrupt-Leitung. Eine Schalteränderung erscheint beim nächsten passenden Read. Ohne Screenshot dieser Version bleibt die Oberfläche schematisch. Ripes wurde hier nicht ausgeführt.
Kontrollfrage: Beweist ein Klick im Tab einen IRQ? Nein. Der Adressvertrag folgt.

Folie 25 / 84

Das Memory-Map-Setup

- Vor dem Code: Base Address der Peripherie ablesen (z. B. `0xF0000000`)
- Falsche Adresse: Schreiben landet im normalen RAM
- Profi-Praxis: Konstanten / BSP-Header

```
#define LED_BASE    0xF0000020
#define SWITCH_BASE 0xF0000000
```

Notizen

Base Address, Offset und ein Board Support Package, kurz BSP, verbinden Karte und Code. Feste typische Adressen sind illustrative Snapshots. Eine falsche Adresse kann RAM, ein anderes Gerät, eine Lücke oder eine Fehlerreaktion treffen. Sie landet nicht zwingend im Heap. Nach einer neuen Geräteanlage müssen die Exports erneut gelesen werden.
Kontrollfrage: Ist 0xF0000000 immer die Schalterbasis? Nein. Damit kann das erste Programm geschrieben werden.

Folie 26 / 84

Feedback-Loop (Das Echo)

- Hello World der Hardware: Schalter spiegelt LED
- Endlosschleife: `lw` Schalter, `sw` LED, Jump

```
# Illustrative Adressen nur, wenn der Export sie bestätigt
li s1, SWITCHES_0_BASE
li s2, LED_MATRIX_0_BASE
Loop:
    lw t0, 0(s1)
    andi t1, t0, 8
    beq t1, zero, Off
    li t2, 0x00FF0000
    j Store
Off:
    li t2, 0
Store:
    sw t2, 0(s2)
    j Loop
```

Notizen

Das Echo schreibt nicht den Rohwert in die Matrix. SW3, Maske 8, wählt Rot 0x00FF0000, sonst Schwarz 0. Andere Schalter ändern diese eine Zelle nicht. Das Programm zeigt einen Pegel. Ein gehaltener Schalter bleibt sichtbar, ohne zu zählen. Die Endlosschleife ist für diese Anzeige beabsichtigt. Das vollständige Programm steht in beispiele/vl8_sw3_rot.s.
Kontrollfrage: Was schreibt 0b1101? Rot, weil Bit 3 gesetzt ist. Zählen macht Prellen relevant.

Folie 27 / 84

Entprellung (Debouncing)

- Mechanische Taster prellen (Bouncing)
- Millisekunden Chaos 1/0, bevor stabil 1
- Pipeline fragt milliardenfach ab
- Ein menschlicher Klick wird als Dutzende Klicks gezählt

Notizen

Mechanisches Prellen ist ein mehrfaches Öffnen und Schließen des Kontakts. Rohsignal, stabiler Zustand und gezähltes Ereignis sind verschieden. Ein Echo ohne Zähler erzeugt nicht von selbst Dutzende Klicks. Eine Maus im Simulator muss nicht prellen. Zeitwerte ohne gemessenen Takt bleiben illustrativ. Die Pipeline fragt nicht allgemein milliardenfach pro Sekunde ab.
Kontrollfrage: Ist jede Flanke ein Klick? Nur wenn die Erkennung sie so zählt. Simulator und reale Kontakte

werden getrennt.

Folie 28 / 84

Software-Debouncing

- Erste 1 erkannt: Druck erkannt
- Entprellung und ein neues Druckereignis sind zwei Schritte
- Eine Schleife mit 10000 Iterationen ist ohne Takt und CPI keine belegte Wartezeit von 20 ms
- Im Simulator: Ereignis plus Release, nicht eine angenommene Millisekundenzahl
- An echter Hardware: Zeitfenster über einen dokumentierten Timer
- Delay-Loop: Takte absichtlich verbrennen

```
li t1, 10000
Delay:
    addi t1, t1, -1
    bne t1, zero, Delay
```

Notizen

10000 Schleifeniterationen sind ohne Takt und CPI, Cycles per Instruction, keine belegte Wartezeit von 20 ms. Für stabile Simulatorpegel genügt Warten auf Druck und danach auf Freigabe. Die Folge 0,0,1,1,1,0,0,1,0 ergibt zwei Ereignisse. Eine Prellfolge 0,1,0,1 kann mehrfach zählen. Eine zeitbasierte Entprellung braucht eine definierte Zeitquelle und eine Stabilitätsdauer. Die wird hier nicht mit einer erfundenen Timeradresse implementiert.
Kontrollfrage: Ist Druck-und-Freigabe schon Entprellung? Nein. Als Nächstes die Frage, wie oft die CPU prüft.

Folie 29 / 84

Polling vs. Interrupts

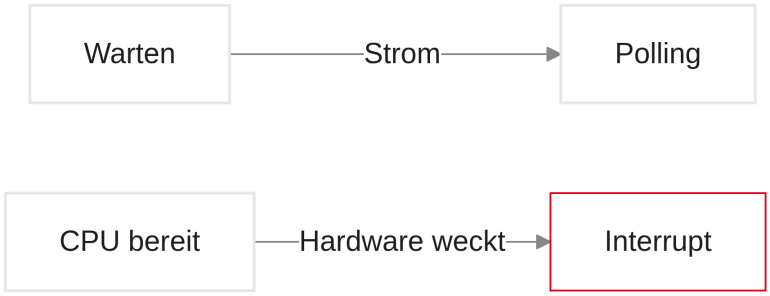
Notizen

Polling ist die programmgesteuerte Abfrage. Ein Interrupt ist ein gemeldetes Ereignis, das unter erfüllten Bedingungen den Ablauf unterbricht. Die Wahl hängt von Häufigkeit, Reaktionszeit, Hardware und Aufwand ab. Die Leitfrage ist, ob zwischen Ereignissen andere Arbeit möglich ist.
Kontrollfrage: Ist Interrupt immer besser? Nein. Zuerst die zwei Wartebilder.

Folie 30 / 84

Aktives Warten vs. Aufwecken

- Ineffizienz im Echo-Design: Busy Waiting
- Polling: Endlosschleife fragt ab
- Interrupt: Hardware weckt die CPU (Türklingel)
- Ausnahmen (Exceptions) vs. Interrupts



Notizen

Busy Waiting wiederholt die Abfrage ohne andere Arbeit. Polling kann auch periodisch zwischen Aufgaben stattfinden. Ein Interrupt kann Arbeit unterbrechen. Schlafen ist keine Voraussetzung. Energie wird nicht automatisch gespart, wenn danach weitergerechnet wird. Exceptions nutzen denselben Trap-Rahmen und folgen später.
Kontrollfrage: Ist jedes Polling Busy Waiting? Nein. Die Größenordnung der Wartezeit folgt.

Folie 31 / 84

Das Problem des Wartens

- Pipeline-CPU z. B. bei 4 GHz: 4 Milliarden Takte pro Sekunde
- Mensch: vielleicht alle 2 Sekunden ein Tastendruck
- Zwischen zwei Klicks: etwa 8 Milliarden Taktzyklen
- Frage: Was tut das Programm in der Zwischenzeit?

```
Ein Mensch ist für einen Prozessor unvorstellbar langsam.
Wartet die CPU auf uns, vergehen für sie "Jahrzehnte".
```

Notizen

4 GHz mal 2 Sekunden sind 8 Milliarden Takte. Das ist ein hypothetisches Beispiel, kein Ripes-Takt. Takte sind nicht automatisch Befehle. Dafür brauchte man CPI und den Schleifenaufbau. In dieser Zeit kann gerechnet, periodisch geprüft oder in einem geeigneten System gewartet werden.
Kontrollfrage: Sind 8 Milliarden Takte 8 Milliarden Abfragen? Nein. Die einfachste Implementierung prüft ununterbrochen.

Folie 32 / 84

Polling (Busy Waiting)

- Naive Lösung: Polling (Abfragen) / Busy Waiting
- Metapher: Kind auf dem Rücksitz — “Sind wir schon da?”
- “Sind wir schon da?” → Nein … Nein … Ja!
- CPU fragt in einer Endlosschleife das MMIO-Register ab, bis Bit = 1

Notizen

Die Schleife liest, prüft und springt zurück. Eine Abfrage ist nicht nutzlos. Eine sehr hohe Frequenz kann im Verhältnis zum Gerät unnötig sein. Für kurze Wartezeiten kann Busy Waiting vertretbar sein. Für lange Pausen blockiert es diesen Programmfluss.
Kontrollfrage: Ist die Rückschleife ein Fehler? Nicht grundsätzlich. Der Assembler macht sie konkret.

Folie 33 / 84

Polling in Assembler

- Wie das Echo-Programm: Wait-Schleife, solange Register = 0
- Erst wenn die Hardware Bit 1 setzt, läuft nützlicher Code weiter

```
li s1, 0xF0000000        # Schalter-Adresse

Wait:
    lw t0, 0(s1)         # MMIO-Read
    beq t0, zero, Wait   # noch 0? sofort wieder

    # Knopf wurde gedrückt - weiter mit Nutzcode
```

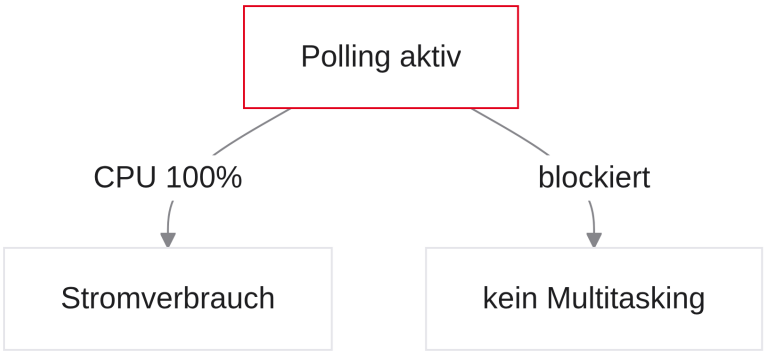
Notizen

Die Adresse kommt aus dem Export. Geprüft wird ein gewähltes Bit, nicht irgendein Rohwert ungleich null. Bitposition 1 ist der Wert 2, nicht dasselbe wie der Zahlenwert 1. Kommt nie ein Ereignis, bleibt die Schleife. Eine gehaltene Taste löst nach dem Verlassen ohne Freigabeprüfung erneut aus.
Kontrollfrage: Was passiert ohne Ereignis? Die Schleife läuft weiter. Die Kosten werden gegen Alternativen gestellt.

Folie 34 / 84

Die Nachteile von Polling

- Busy Waiting ist eine Variante: die CPU prüft ununterbrochen
- Periodisches Polling in einer sonstigen Anwendung ist weiterhin üblich
- Es ist nicht gleichbedeutend mit „kein Multitasking“



Notizen

Periodisches Polling ist legitim. Die Grafik darf nicht jedes Polling als Blockade zeichnen. Busy Waiting, periodische Abfrage und Interrupt unterscheiden sich in Aufwand, Reaktionszeit und nötiger Hardware. Bei periodischer Abfrage kann die zusätzliche Wartezeit etwa eine Periode betragen, plus Bearbeitung.
Kontrollfrage: Verhindert Polling Multitasking? Nicht grundsätzlich. Eine Gerätemeldung kann die Nachfrage ersetzen.

Folie 35 / 84

Die Hardware-Lösung: Die Türklingel

- Paket erwarten: nicht stündlich an der Klinke rütteln (Polling)
- Stattdessen: Türklingel einbauen
- Normales Leben weiterführen; erst beim Klingeln reagieren
- Rolle umkehren: Peripherie meldet sich, wenn Daten da sind

Notizen

Die Klingel ist nur die Meldung. Die Bearbeitung ist die Routine. Eine IRQ, Interrupt Request, braucht Aktivierung, Entscheidung und einen Handler. Nicht jede Peripherie und nicht die Ripes-Schalter besitzen automatisch eine Interrupt-Leitung.
Kontrollfrage: Verarbeitet die Klingel das Paket? Nein. Der Begriff Interrupt folgt.

Folie 36 / 84

Einführung von Interrupts

- Türklingel-Prinzip = Interrupt (Unterbrechung)
- Spezielle Pins verbinden Peripherie mit der CPU
- Tastendruck: Stromimpuls auf den Interrupt-Pin
- CPU speichert Kontext, springt zur ISR, kehrt danach zurück



Notizen

Ein Interrupt kann eine Flanke, ein Pegel, ein Controller oder ein interner Timer sein, nicht nur ein Stromimpuls. Im vereinfachten M-Mode-Modell zählen Pending, Quellenfreigabe und das globale MIE, Machine Interrupt Enable. Die Hardware rettet nicht alle x-Register. Ein Taster-IRQ wird nur versprochen, wenn das Gerät ihn nachweislich hat. Das ist hier nicht belegt.
Kontrollfrage: Sichert der Trap t0? Nein. Der Zeitablauf folgt.

Folie 37 / 84

Der Ablauf eines Interrupts

- Main-Programm läuft (z. B. Berechnung / Player)
- Interrupt-Signal trifft den CPU-Pin
- Hardware unterbricht; PC springt zur ISR
- ISR verarbeitet die Eingabe
- `mret`: Rückkehr ins Main-Programm



Notizen

Der Ablauf ist Nutzprogramm, angenommene Anforderung, Hardwareeintritt, Kontextrettung, Behandlung, Wiederherstellung und Rückkehr. Zwischen Ereignis und Handler kann Latenz liegen. mret gilt für den M-Mode-Handler dieses Modells. Die Quittierung hängt vom Gerät ab. Bei einem Interrupt wird mepc nicht pauschal um 4 erhöht.
Kontrollfrage: Beginnt die ISR im selben Moment wie der Tastendruck? Nicht zwingend. Nicht jede Unterbrechung ist asynchron.

Folie 38 / 84

Exceptions vs. Interrupts

	Exceptions	Interrupts
Quelle	CPU-intern	Hardware-extern
Auslöser	aktueller Befehl	unabhängig vom Code
Timing	synchron	asynchron
Beispiel	Illegal Instr., Access Fault	Timer, Tastendruck

Notizen

Eine Exception ist einem Befehl zugeordnet. Ein Interrupt ist gegenüber diesem Befehlsstrom asynchron. Intern und extern als einzige Einteilung ist falsch, etwa bei einem Timer oder einem Speicherfehler. ecall ist eine absichtliche Exception, kein Fehler.
Kontrollfrage: Ist jeder Trap ein Fehler? Nein. Typische Exceptions folgen.

Folie 39 / 84

Exceptions im Detail

- Illegal Instruction: Decoder kennt den Opcode nicht
- Access Fault: `lw`/`sw` auf gesperrte/ungültige Adresse (ohne MMU)
- Page Fault: nur mit virtuellem Speicher (Paging)
- Hinweis: Integer-Div/0 trappt in RISC-V nicht (anders als x86)
- OS beendet oft mit Segmentation Fault

```
int *ptr = NULL;
int crash = *ptr;   /* typisch: Access Fault */
```

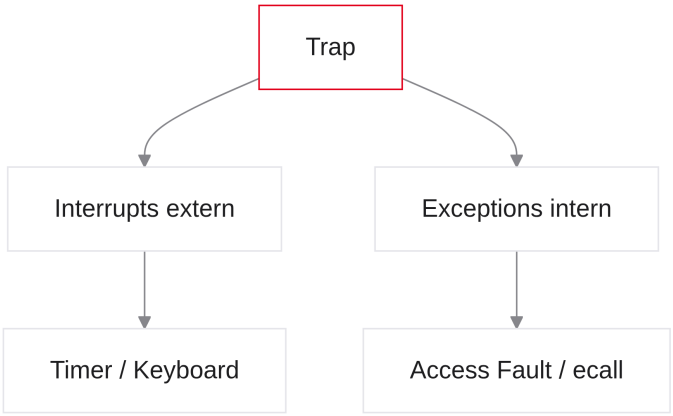
Notizen

Illegal Instruction kann auch ein unerlaubter CSR-Zugriff sein, nicht nur ein unbekannter Opcode. Ein Access Fault ist nicht nur ein System ohne MMU. MMU bedeutet Memory Management Unit. Ein Page Fault betrifft Übersetzung oder Rechte. Ein Nullzeiger in C hat undefiniertes Verhalten und ist kein garantierter Access Fault. Adresse 0 kann im Simulator gültig sein. Eine Integerdivision durch null in der M-Erweiterung erzeugt keinen Trap. Der Quotient ist definiert, nicht eine Exception.
Kontrollfrage: Stürzt Division durch null in RV32M ab? Nein. Beide Klassen heißen Traps.

Folie 40 / 84

Der Regenschirm-Begriff: Traps

- Exceptions und Interrupts teilen denselben Rettungsablauf
- RISC-V-Dachbegriff: Trap (Falle)
- Trap = Unterbrechung des Kontrollflusses ohne Branch/Jump
- “Trap ausgelöst” = Notfall-Ablauf gestartet



Notizen

Ein Trap übergibt den Kontrollfluss wegen Exception oder Interrupt. Ein Syscall ist eine reguläre Anfrage, kein Notfall. Auch Firmware ohne OS kann Traps behandeln. Ein Branch ist gewöhnlicher Programmfluss. Der Trap-Eintritt wird durch das Ereignis bestimmt.
Kontrollfrage: Braucht jeder Trap Linux? Nein. Der Hardwarezustand folgt.

Folie 41 / 84

Was passiert bei einem Trap?

1. Rettungsanker: aktuellen PC merken (Rückkehr)
2. Pipeline flushen: IF/ID hinter dem Ereignis löschen
3. Sprung: PC auf feste Handler-Adresse setzen (ISR)

```
Ein Trap ist ein erzwungener Hardware-Sprung
an eine vom Betriebssystem vorbereitete Adresse.
```

Notizen

Im benannten M-Mode-Modell werden Rückkehradresse, Ursache und Status festgehalten. Der Handler liegt am konfigurierten Ziel. Ein Pipeline-Flush kann jüngere Wirkungen unterdrücken, ist aber nicht die Definition des Traps. Nur IF und ID zu löschen genügt nicht allgemein. IF und ID bedeuten Instruction Fetch und Decode. Das Ziel hängt vom Vector-Modus ab.
Kontrollfrage: Löscht der Trap nur IF/ID? Nicht als allgemeine Regel. Die Registerrettung bleibt Software.

Folie 42 / 84

Die Gefahr des Clobberings

- ISR ist normaler Assembler und nutzt `t0`, `s1`, `a0`, ...
- Trap kommt ohne Vorwarnung mittendrin
- Die ABI gilt weiter — aber es gibt keinen Caller, der Temporaries rettet
- Deshalb: Handler muss jedes genutzte Register selbst sichern

Notizen

Clobbering heißt, einen noch benötigten Registerwert zu überschreiben. Hat das Hauptprogramm t0 gleich 7 und der Handler setzt t0 auf 99, rechnet das Programm falsch weiter. Die Funktions-ABI, Application Binary Interface, gilt für normale Aufrufe. Ein asynchron unterbrochenes Programm konnte seine temporären Register nicht vorher sichern. Ein CSR ist kein Backup der Registerbank.
Kontrollfrage: Rettet die Hardware t0? Nein. Die CSRs liefern Ursache und Rückkehradresse.

Folie 43 / 84

Das CSR-System

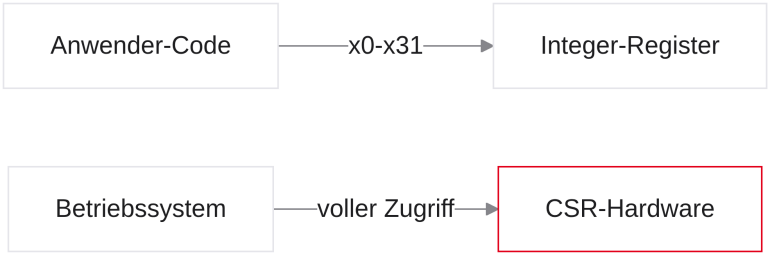
Notizen

Ein CSR ist ein Kontroll- oder Statusregister, kein gewöhnliches x-Register. Wir brauchen Ursache, Rückkehrstelle, Einsprungziel und Freigabestatus. Schattenregister darf nur Metapher sein, keine zweite Kopie der Registerbank. Zicsr ist die Erweiterung für die Zugriffsbefehle, ohne erfundene Langform des Kürzels.
Kontrollfrage: Liegt mcause in x31? Nein. Der Bedarf an diesem Zustand folgt.

Folie 44 / 84

Schattenregister für die Notaufnahme

- Zustand retten, ohne User-Register zu zerstören
- `x0–x31` sind durch Anwendercode belegt
- Privilegienstufen (Ring-Sicherheit)
- CSRs: unsichtbarer Speicherplatz im Chip
- Befehle `csrr`, `csrw` und der ISR-Ablauf



Notizen

CSRs halten Ursache, Rückkehradresse und Statusregeln. Anwendercode kann einzelne freigegebene CSRs lesen. M-Mode ist kein x86-Ring. Die Hardware sichert keine Anwendungsdaten automatisch. Der Handler muss Register selbst erhalten.
Kontrollfrage: Sind CSRs unsichtbar für jeden User-Zugriff? Nicht pauschal. Der Stackunterschied folgt.

Folie 45 / 84

Die Beschränkung von RV32I

- RV32I: genau 32 Integer-Register — vom Compiler ausgereizt
- OS braucht Platz für Timer, Trap-Ursache, Puffer — oft vor sicherem Stack-Zugriff
- Teufelskreis: Stack-Sicherung braucht `sp` — dem User-`sp` darf man nicht trauen

```
Schatten-Register: vertrauenswürdig und nicht manipulierbar.
```

Notizen

RV32I hat 32 Architekturregister. x0 bleibt null. Nicht jeder Compiler nutzt alle. Timer und Puffer liegen gewöhnlich nicht als Daten in der CSR-Bank. Ein Handler darf einen vom User kontrollierten Stackzeiger sp nicht ungeprüft als privilegierten Stack verwenden. CSR-Zugriffe folgen Rechten, nicht einer absoluten Unmanipulierbarkeit.
Kontrollfrage: Darf der Handler den User-sp blind nutzen? In diesem U-nach-M-Modell nein. Der Adressraum der CSRs folgt.

Folie 46 / 84

Die CSRs (Control & Status Registers)

- Extension Zicsr: separater Adressraum, bis zu 4096 CSRs
- Kein RAM/MMIO — Flip-Flop-Bänke im Kern
- Namen z. B. `mstatus`, `mtvec`, `mcause`
- Normale Befehle (`add`, `lw`) greifen nicht zu

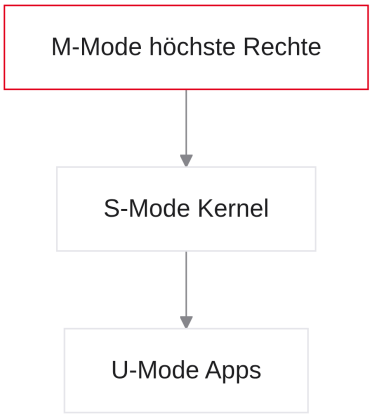
Notizen

Ein 12-Bit-Spezifikator erlaubt bis zu 4096 Adressen, nicht 4096 vorhandene Register. CSRs sind weder RAM noch MMIO. Lesen, Schreiben und zulässige Werte hängen vom einzelnen Register ab. Nicht alle sind für User-Code unsichtbar. Geeignete Zähler können lesbar sein.
Kontrollfrage: Ist 4096 die Zahl der Flip-Flops? Nein. Der Modus entscheidet den Zugriff.

Folie 47 / 84

Privilegienstufen (Privilege Levels)

- Machine Mode (M): höchste Rechte, Boot; voller CSR-Zugriff (Firmware)
- Supervisor Mode (S): großes OS (Linux); viele CSRs (z. B. Paging)
- User Mode (U): Anwendungen; unerlaubte privilegierte CSR-Zugriffe erzeugen einen Trap



Notizen

M-Mode ist die fundamentale privilegierte Stufe. Supervisor und User müssen nicht auf jedem kleinen System existieren. Voller M-Zugriff erlaubt trotzdem kein beliebiges Schreiben in nur lesbare oder fehlende CSRs. Ein unerlaubter Zugriff erzeugt Illegal Instruction. Speicherschutz gegen MMIO entsteht durch Rechte oder Übersetzung, nicht allein durch den Namen User Mode.
Kontrollfrage: Darf User-Code nie einen CSR lesen? Doch, wenn er freigegeben ist. Die Zugriffsbefehle folgen.

Folie 48 / 84

CSR-Befehle (Zicsr)

- `csrr rd, csr`: CSR → normales x-Register
- `csrw csr, rs1`: x-Register → CSR

```
csrr t0, mstatus     # Status nach t0
li t1, 8             # Maske: Bit MIE
csrrs x0, mstatus, t1  # nur Bits setzen (OR)
```

Notizen

csrr ist die Kurzform von csrrs mit Quelle x0. csrw ist csrrw mit Ziel x0. MIE ist Bit 3, die Maske ist 8, nicht 1. csrrs setzt die maskierten schreibbaren Bits und erhält die anderen. Ein csrw kann das ganze Register ersetzen. Vor dem Einschalten müssen Handler und Quellen konfiguriert sein.
Kontrollfrage: Setzt die Maske 1 das Bit MIE? Nein, Maske 8. Der kombinierte Zugriff vermeidet Zwischenzustände.

Folie 49 / 84

Atomare CSR-Befehle

- Gefahr: Interrupt zwischen `csrr` und `csrw` (Race Condition)
- Lösung: `csrrw` — Read und Write in einer Instruktion (atomar)
- Software-seitig ununterbrechbar (eine Instruktion)

```
# Lehrbeispiel: nur MIE (Bit 3, Maske 8) setzen, übrige Bits erhalten
li t1, 8
csrrs x0, mstatus, t1
```

Notizen

csrrw tauscht den ganzen Wert. csrrs setzt Bits, csrrc löscht Bits. mstatus mit 1 zu überschreiben wäre falsch, weil andere Statusbits verloren gingen. Für MIE ist csrrs mit Maske 8 passend. Die Atomarität einer CSR-Instruktion ist keine Mehrkernsperre und keine Speicherbarriere.
Kontrollfrage: Erhält csrrw automatisch alle übrigen Bits? Nein. Die Trap-Register folgen.

Folie 50 / 84

CSR: mtvec (Trap-Vector)

- Vier Kern-CSRs für Trap-Handling
- `mtvec`: BASE und MODE
- Direct: Trapziel ist BASE, nicht eine blinde Kopie des ganzen Registers

```
Boot des OS:
  la t0, MeineRettungsRoutine
  csrw mtvec, t0
```

Notizen

mtvec enthält BASE und MODE. Im Direct-Modus gehen alle Traps nach BASE. Im Vectored-Modus gehen Interrupts nach BASE plus 4 mal Ursache, Exceptions weiterhin nach BASE. Die Hardware kopiert nicht blind das ganze Register in den PC. Das Label muss ausgerichtet sein. Ob Venus diesen CSR-Zugriff ausführt, ist nicht belegt.
Kontrollfrage: Ist das Ziel immer exakt der Registerinhalt? Nein, es hängt vom Modus ab. Die Fortsetzungsstelle folgt.

Folie 51 / 84

CSR: mepc (Exception PC)

- Wo wurde der User-Code unterbrochen?
- Hardware kopiert den unterbrochenen PC automatisch nach `mepc`
- Bei synchronen Exceptions/`ecall`: `mepc` zeigt auf den auslösenden Befehl
- `mepc + 4` nur, wenn ein 4-Byte-`ecall` konsumiert wird
- Bei einem Interrupt die Fortsetzungsadresse in `mepc` nicht pauschal erhöhen

Notizen

Bei einem Interrupt bleibt die gespeicherte Fortsetzungsadresse. Ein bewusst konsumierter 32-Bit-ecall wird um 4 erhöht, weil der Befehl selbst nicht wiederholt werden soll. Ein zu wiederholender Fehler behält die Fehleradresse. ecall bleibt auch mit C-Erweiterung 32 Bit lang. Venus verbirgt diese Fortsetzung und beweist damit kein sichtbares mepc.
Kontrollfrage: Darf jeder Handler mepc um 4 erhöhen? Nein. Die Ursache entscheidet die Strategie.

Folie 52 / 84

CSR: mcause (Cause)

- Warum aufgeweckt? Tastendruck, Access Fault, Syscall?
- Hardware schreibt Fehler-/Ursachencode nach `mcause`
- MSB = 1: Interrupt; MSB = 0: Exception (zuerst prüfen)
- Beispiele: 2 = Illegal Instruction; 8 = Environment Call (U-Mode)

```
csrr t0, mcause
li t1, 8
beq t0, t1, Syscall   # nach Interrupt-Bit-Check
```

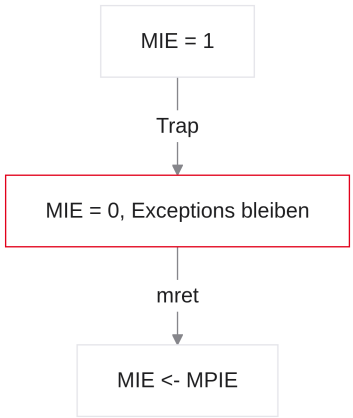
Notizen

Bei RV32 ist Bit 31 das Interrupt-Bit. Der übrige Code ist die Ursache. 0x00000008 ist U-ecall, Ursache 8. 0x0000000B ist M-ecall, Ursache 11. 0x8000000B ist ein External Interrupt mit gesetztem Interrupt-Bit und demselben niedrigen Code 11. Der Code ersetzt nicht die Dienstnummer des Syscalls.
Kontrollfrage: Bedeutet Code 11 immer denselben Trap? Nein, das Interrupt-Bit unterscheidet. mstatus begleitet Eintritt und Rückkehr.

Folie 53 / 84

CSR: mstatus (Status)

- Herz der Kontrolle; u. a. Bit MIE (Machine Interrupt Enable)
- MIE = 1: maskierbare M-Interrupts können angenommen werden
- MIE = 0: diese Interrupts sind global gesperrt; synchrone Exceptions bleiben möglich
- Beim Trap: Hardware setzt MIE automatisch auf 0 (kein Nesting)



Notizen

MIE sperrt oder erlaubt maskierbare M-Interrupts im aktuellen M-Mode. Quellenfreigabe und Pending bleiben nötig. MIE gleich 0 verhindert keine synchronen Exceptions. Beim Eintritt gilt MPIE gleich bisheriges

MIE, MIE gleich 0 und MPP gleich vorheriger Modus. MPIE ist Machine Previous Interrupt Enable, MPP Machine Previous Privilege. Previous ist der gespeicherte Zustand für die Rückkehr. Dieses Lehrmodell schaltet im Handler nicht erneut frei.
Kontrollfrage: Macht MIE gleich 0 die CPU für ecall taub? Nein. Der Handler-Prolog folgt.

Folie 54 / 84

Der Trap-Handler (ISR)

- Schritt 1: Kontextsicherung aller genutzten x-Register
- Echter Kernel: zuerst `sp` mit `mscratch` tauschen (User-`sp` ist unsicher)
- Unten: vereinfachte Lehr-Sequenz (Stack bereits gültig angenommen)
- `mtvec` enthält BASE und MODE; dieser Kurs nimmt Direct an
- Nicht jeder CSR ist im User Mode verboten
- MIE ist der globale Machine-Interrupt-Enable, nicht die gesamte Konfiguration
- Einzelne Quellen und Pending-Bits kommen dazu

```
Trap_Handler:
    # Lehrannahmen: gültiger ausgerichteter Stack, keine verschachtelten Traps
    # Leaf-Handler: nur die selbst benutzten Register
    addi sp, sp, -16
    sw   t0, 12(sp)
    csrr t0, mcause
    # Quelle quittieren, ohne weiteren Call
    lw   t0, 12(sp)
    addi sp, sp, 16
    mret
```

Ein Handler mit Unteraufruf muss `ra` und alle möglicherweise veränderten Caller-Saved-Register sichern. Für `ra`, `t0–t6` und `a0–a7` sind das 16 Wörter, also 64 Byte. `mepc` wird nur bei einem bewusst konsumierten 4-Byte-`ecall` um 4 erhöht. Ein Interrupt setzt bei `mepc` bereits die Fortsetzungsadresse; dort nicht pauschal addieren.

Notizen

Annahmen: Direct-Modus, gültiger 16-Byte-ausgerichteter Stack, kein verschachteltes Trap, Leaf ohne call. Der Frame von 16 Byte sichert t0. csrr liest die Ursache. Danach wird t0 zurückgeladen und mret ausgeführt. Eine Quittierung ohne Gerätevertrag ist nur ein Gerüst. Ein U-nach-M-Handler kann sp mit mscratch tauschen. Das ist eine mögliche Konvention, kein universelles OS-Rezept.
Kontrollfrage: Darf dieser Leaf-Handler call ausführen? Nein, ra wäre ungesichert. Ein Unteraufruf vergrößert den Frame.

Folie 55 / 84

Kontext-Sicherung und Arbeit

- Schritt 2: `mcause` lesen, Treiber, eigentliche Aufgabe
- Schritt 3: Register spiegelverkehrt wiederherstellen

```
# Nicht so: call zerstört ra und weitere Caller-Saved-Register,
# die der 16-Byte-Leaf-Frame oben nicht gesichert hat.
# Entweder Leaf ohne Call, oder 64-Byte-Frame für ra, t0-t6, a0-a7.
```

Notizen

call verändert ra und kann t- und a-Register verändern. Unter der üblichen RV32-Integerkonvention sind ra, t0 bis t6 und a0 bis a7 sechzehn Wörter, also 64 Byte. Die Ausrichtung bleibt 16 Byte. s-Register bleiben nur erhalten, wenn die Konvention eingehalten wird. Gleitkomma liegt außerhalb. mepc wird nur nach begründeter Policy geändert. Ein IRQ-Handler muss t0 erhalten, obwohl t0 bei normalen Funktionen caller-saved ist, weil der unterbrochene Code nicht sichern konnte.
Kontrollfrage: Warum t0 sichern, wenn es caller-saved ist? Weil kein Aufruf voranging. Danach die Rückkehr.

Folie 56 / 84

Die Rückkehr (mret)

- Nicht `ret` — in `ra` steht nichts Brauchbares
- Zieladresse liegt in `mepc`
- `mret` (Machine Return) atomar:
  - `mepc` → PC
  - MIE wird aus MPIE wiederhergestellt (oft wieder 1)
  - Privilegien zurück (oft U-Mode)

```
    mret    # zurück ins User-Programm
```

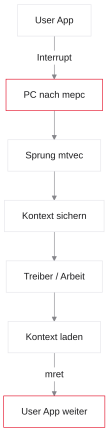
Notizen

ret springt über ra. mret stellt PC aus mepc her und den Modus aus MPP. In ra kann eine gültige Funktionsadresse liegen, nur nicht die Trap-Fortsetzung. Bei M nach M wird nicht nach User abgestuft. MIE wird aus MPIE wiederhergestellt. Danach setzt das Referenzmodell MPIE auf 1 und MPP auf den niedrigsten unterstützten Modus. Das ist der gespeicherte Wert, nicht der gerade wiederhergestellte aktuelle Modus. Ein S-Mode-Handler nutzt sret.
Kontrollfrage: Setzt mret immer nach User zurück? Nein. Der Gesamtablauf folgt.

Folie 57 / 84

Zusammenfassung Trap-Ablauf

- Lebenszyklus jedes Multitasking-OS
- Ein Kern: scheinbar viele Programme + I/O ohne Absturz



Notizen

Ein vollständiger Durchlauf ist Ereignis, Hardwareeintritt, Rettung, Behandlung, Wiederherstellung und mret. Das ist nicht automatisch ein Prozesswechsel. Timer können Scheduling auslösen, müssen es aber nicht. Die Kontrollfrage lautet: Warum t0 sichern, obwohl Funktionen es nicht erhalten? Weil der Interrupt asynchron kommt. Die Antwort steht in der vorherigen Notiz.
Kontrollfrage: Wechselt jeder Trap den Prozess? Nein. Dieselbe Idee dient absichtlichen Diensten.

Folie 58 / 84

Syscalls & Das Betriebssystem

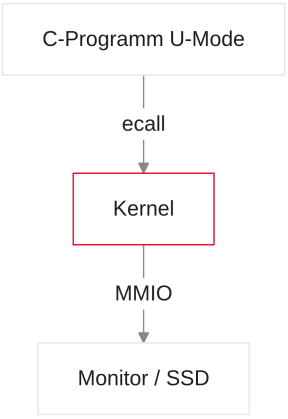
Notizen

Ein System Call ist eine Dienstanforderung. ecall löst sie aus. Das Laborziel ist Textausgabe. Venus erhält dafür keinen vollständigen privilegierten Kernel. Der bisherige Trap erklärt den Rahmen, der Dienst den Zweck.
Kontrollfrage: Ist die Konsolenausgabe schon ein Kernel? Nein. Die Schnittstelle folgt.

Folie 59 / 84

Absichtliche Traps und der Pförtner

- Software kann absichtlich einen Trap auslösen
- Mauer zwischen User und Hardware
- OS als Pförtner
- Befehl `ecall` und ABI-Verträge
- Labor: Text auf die Venus-Konsole



Notizen

Die Anwendung fordert einen Dienst an. Die Umgebung prüft Argumente und Rechte, führt die Operation aus und kann ein Ergebnis liefern. Das ist kein Notfall. Die ABI ist der binäre Vertrag über Register und Parameter. Ein API, Application Programming Interface, benennt die angebotenen Operationen. Venus simuliert den Dienst, ein echter User-nach-Kernel-Übergang ist ein anderes Modell.
Kontrollfrage: Ist der Pförtner ein Notarzt? Nein. Als Nächstes der absichtliche Trap.

Folie 60 / 84

Software-Traps

- Hardware kann die CPU per Interrupt unterbrechen
- Kann ein User-Programm absichtlich einen Trap auslösen?
- Ja: wenn C-/Assembler-Code die Hilfe des OS braucht
- Absichtliche Software-Traps = System Calls (Syscalls)

```
Ein Syscall ist wie ein roter Notfallknopf.
Das Programm drückt ihn, friert sich ein und ruft das OS zu Hilfe.
```

Notizen

ecall fordert einen Dienst an. ebreak dient eher dem Debugging. Nicht jeder absichtliche Trap ist ein Syscall. Die Ausführung geht kontrolliert über, das Programm friert nicht ein. Ein Dienst kann später

blockieren, muss es aber nicht. Direkter Gerätezugriff ist in Bare Metal üblich.
Kontrollfrage: Ist ebreak ein Print? Nein. Geschützte Anwendungen überlassen Geräte oft der Umgebung.

Folie 61 / 84

Warum wir das OS brauchen

- Warum nicht einfach `sw` direkt auf den Monitor?
- Sicherheit: freier MMIO-Zugriff könnte die Platte zerstören; User-Mode sperrt MMIO hardwareseitig
- Abstraktion: tausende Monitore/Treiber — das OS kapselt Hardware in einer universellen Schnittstelle

Notizen

Schutz, Verwaltung und Abstraktion erklären eine Konsole. User Mode sperrt MMIO nicht automatisch. Dafür braucht es Rechte oder Übersetzung. Ein OS ist für viele Systeme typisch, für jeden eingebetteten Rechner aber nicht nötig. printf formatiert meist in einer Bibliothek und nutzt dann einen Ausgabedienst.
Kontrollfrage: Ist jeder printf ein direkter Gerätestore? Nein. Der Kern stellt den Zugang bereit.

Folie 62 / 84

Der Pförtner (Das Betriebssystem)

- OS im privilegierten Ring (S- oder M-Mode)
- Verwaltet Hardware exklusiv
- Agiert als Pförtner / Torwächter
- Drucken = Formular ausfüllen, an die Kernel-Tür klopfen
- Diese Formulare sind System Calls



Notizen

Ein Kernel und seine Treiber arbeiten privilegiert. RISC-V nutzt Modi, keine x86-Ringe. Der Vertrag prüft Nummer, Zeiger und Rechte. Der Kernel muss nicht jedes Gerät allein bedienen. Für den Einstieg genügt ein vereinfachtes geschütztes Modell.
Kontrollfrage: Ist M-Mode Ring 0? Die Rollen ähneln sich, die Namen nicht. Der Auslösebefehl folgt.

Folie 63 / 84

Der Befehl ecall

- RISC-V-Klopfen an die Kernel-Tür: `ecall` (Environment Call)
- Keine Operanden — steht nackt im Code
- Decoder liest `ecall` → sofort Trap-Handler
- Hardware-erzwungener Kontextwechsel

```
ecall    # aus U-Mode in den Kernel
```

Notizen

ecall hat keine Operanden. Die Parameter stehen vorher in Registern. Die ISA kennt Print Integer nicht. Die Umgebung definiert den Dienst. Ein Trap ist kein automatischer Prozesswechsel. Ein Simulator kann die Anfrage im Host behandeln.
Kontrollfrage: Steht die Dienstnummer im Befehlswort von ecall? Nein, in einem Register. Die Liste der Dienste folgt.

Folie 64 / 84

System Calls (Die API des OS)

- Hunderte Dienste: Datei, Netz, Zeit, Text drucken, …
- Diese Liste = System Call API
- Frage nach `ecall`: Welcher Service?
- Antwort: Vertrag erfüllen — die Syscall-ABI

Notizen

Die API nennt die Operation, die ABI Nummer, Register und Fehlerregel. Hunderte Dienste sind eine mögliche OS-Eigenschaft, nicht die drei Venus-Dienste dieses Labors. Die Speisekarte braucht diese Zuordnung, sonst bleibt sie eine Metapher.
Kontrollfrage: Ist Print Integer Teil der RISC-V-ISA? Nein. Die Register unterscheiden sich je Umgebung.

Folie 65 / 84

Die Syscall-ABI (Venus vs. Linux)

- Vor `ecall` muss eine Service-ID gesetzt werden — Registerkonvention ist ABI-abhängig
- Venus (Labor): ID in `a0`, Argumente in `a1`, `a2`, …
- Linux / RARS: ID in `a7`, Argumente in `a0`, `a1`, …
- Im Praktikum: Venus-Konvention verwenden

```
Venus:  a0 = Service-ID,  a1 = Argument
Linux:  a7 = Service-ID,  a0 = Argument
```

Notizen

Venus: Nummer in a0, erstes Argument in a1. Ripes und RARS: Nummer in a7, erstes Argument in a0. Linux auf RISC-V: Nummer in a7, Argumente in a0 bis a5. Gleiche Register sind nicht gleiche Dienste. Print Integer 1 ist ein Simulatordienst, kein Linux-Dienst. MIPS-Konventionen mit v0 bleiben außen vor. Kontrollfrage: Darf Venus-Code a7 gleich 1 für die Ausgabe nutzen? Nicht in dieser Konvention. Der Venus-Vertrag folgt im Code.

Folie 66 / 84

Code-Beispiel: Print Integer (Venus)

- Service 1 in `a0`, Zahl 42 in `a1`, dann `ecall`
- Venus kehrt danach zur nächsten Instruktion zurück (Handler intern)

```
li a0, 1          # Venus: Service Print Integer
li a1, 42         # Argument
ecall
```

Notizen

li a0, 1 setzt die Dienstnummer. li a1, 42 setzt den Wert, keine Adresse. ecall fordert die Ausgabe 42 an. li ist ein Pseudobefehl. Kleine Konstanten passen in eine Immediate-Operation. Die Fortsetzung stellt Venus bereit. Nicht jeder Bare-Metal-Handler muss mepc erhöhen. Das gilt nur für einen bewusst konsumierten ecall im passenden Trap-Modell.
Kontrollfrage: Ist 42 eine Stringadresse? Nein. Danach das Programmende.

Folie 67 / 84

Code-Beispiel: Exit Program (Venus)

- Endlosschleife am Code-Ende ist unsauber
- Service 10 = Exit; keine Argumente; keine Rückkehr

```
li a0, 10        # Venus: Exit
ecall
```

Notizen

Dienst 10 beendet ohne Rückkehr. Eine Anweisung dahinter ist kein sicherer Programmschritt. Eine Endlosschleife ist für dauerhafte Firmware legitim. Für diese Konsole ist Exit der klare Abschluss. Das minimale Programm gibt aus und beendet.
Kontrollfrage: Läuft Code nach Exit zuverlässig weiter? Nein. Simulator und privilegierte Architektur werden getrennt.

Folie 68 / 84

Was macht ecall unter der Haube?

- Decoder liest `ecall` → Hardware schaltet:
  - PC des `ecall` → `mepc`
  - `mcause` = 8 (Environment Call from U-Mode)
  - PC → Adresse in `mtvec`
- Handler: Ursache 8 → Syscall; Service-ID laut ABI lesen; oft `mepc += 4`



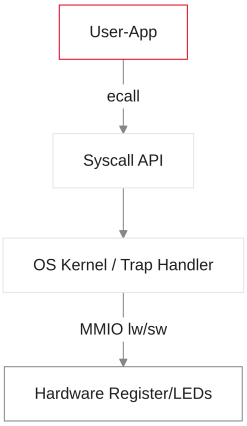
Notizen

Im U-nach-M-Modell ohne Delegation, Direct-Modus, gilt mepc gleich ecall-Adresse, mcause gleich 8 und das Ziel BASE. Ein M-ecall hätte Ursache 11. Ein konsumierter ecall wird um 4 erhöht. Linux mit Supervisor nutzt stvec, sepc und sret. Venus erklärt den Dienst, ohne diese CSRs als nachgewiesene Simulation zu behaupten.
Kontrollfrage: Ist mcause bei jedem ecall 8? Nur aus User Mode in diesem Modell. Das Schichtenbild greift das auf.

Folie 69 / 84

Das Betriebssystem-Modell

- Schichten-Modell: Abstraktion schützt vor harter Physik



Notizen

Ein Ausgabewert geht durch Anwendung, Bibliothek, Dienst und Gerät. Ergebnisse und Fehler können zurückkommen. Abstraktion und Schutz sind verschieden. Venus abstrahiert den Konsolendienst. Die sichtbare Ausgabe beweist keinen Kernel und kein MMIO-Register.
Kontrollfrage: Folgt aus der Konsole ein mtvec? Nein. Beide Modelle gehen ins Labor.

Folie 70 / 84

Praxis: Ripes & Venus

Notizen

Zwei Ziele: in Ripes eine Maske in eine Farbe übersetzen, in Venus einen String und einen berechneten Wert ausgeben. mret ist keine Voraussetzung. Ripes prüft Adressen und Farbwort, Venus die Register vor ecall.
Kontrollfrage: Braucht Hello World einen Trap-Handler? In Venus nein. Die Werkzeugwahl folgt.

Folie 71 / 84

Dualer Laboreinsatz

- Wechsel von Theorie zum Labor
- Ripes vs. Venus: unterschiedliche Zwecke
- Ripes: virtuelle MMIO-Hardware (LEDs)
- Venus: Konsole über `ecall`
- Typische I/O-Bugs und Clobbering-Warnung

Notizen

Ripes zeigt hier die Matrix, Venus die Konsole. Beide Werkzeuge können mehr. Ripes muss nicht geschlossen werden. Erfolg in Ripes ist die erwartete Farbe, in Venus die erwartete Ausgabe.
Kontrollfrage: Schließt das eine Werkzeug das andere aus? Nein. Die Grenzen werden benannt.

Folie 72 / 84

Tool-Zweck im Labor

- Zwei Welten: nackte Hardware (MMIO) vs. Abstraktion (Syscalls)
- Ripes: MMIO-Labor mit Switches und LED-Matrix; Environment Calls existieren zusätzlich
- Venus: Instruktionssimulator mit Konsolendiensten über ecall, kein nachgewiesener Kernel

```
Ripes  → Schalter-LED-Labor
Venus  → Konsolenausgabe mit ecall
```

Notizen

Ripes ist für dieses Labor das MMIO-Werkzeug und besitzt zusätzlich Environment Calls. Venus ist ein Instruktionssimulator mit Konsolendiensten, kein nachgewiesener OS-Kernel. Version und Prozessormodell wurden hier nicht live geprüft. CSR-Zugriffe und mret werden nicht als in beiden Werkzeugen lauffähig behauptet.
Kontrollfrage: Simuliert Venus mtvec? Das ist nicht belegt. Die Geräte werden angelegt.

Folie 73 / 84

Ripes I/O-Tab Setup

- Reiter I/O in Ripes öffnen
- Plus: Switches und LED Matrix hinzufügen
- Ripes verdrahtet virtuell und vergibt MMIO-Adressen

Notizen

Switches mit mindestens vier Eingängen und eine LED-Matrix, für das Lauflicht eine Zeile und acht Spalten, werden im I/O-Tab angelegt. Registerkarte und Exports müssen abgelesen werden. Interaktives Run ist keine Messung realer Latenz. Ein geänderter Aufbau kann die Adressen ändern. Ein UI-Test dieser Fassung liegt nicht vor.
Kontrollfrage: Bleiben die Adressen nach dem Umbau gleich? Nicht garantiert. Der Code muss sie verwenden.

Folie 74 / 84

Ripes: MMIO-Adressen prüfen

- Bauteil anklicken → Konfiguration unten links
- Base Address zwingend ablesen
- Base Address aus der Registerkarte bzw. den Exports ablesen
- Die Zahlen unten sind ein illustrativer Snapshot, keine universelle Hardwarekonstante

Gerät	Typ	Illustrativer Snapshot
Switches	Input (Bit-Map)	Export `SWITCHES_0_BASE`
LED Matrix	Output (Wort-Array)	Export `LED_MATRIX_0_BASE`

Notizen

SWITCHES_0_BASE und LED_MATRIX_0_BASE sind die Namen, sofern der Export sie so liefert. Eine Größe in Byte ist nicht automatisch die Pixelzahl. Ein Wort je Pixel bedeutet Teilung durch 4. Ein fester Snapshot wie 0xF0000000 bleibt illustrativ. Zugriffstyp und Breite werden in der Karte geprüft.
Kontrollfrage: Ist LED_MATRIX_0_SIZE immer die Pixelzahl? Nein. Der erste Pixel folgt.

Folie 75 / 84

Labor-Aufgabe 1 (Ripes)

- Hello Hardware: Endlosschleife liest Switches
- Mit `and`-Maske prüfen: Schalter 3 an?
- Ja: erste LED rot (`0x00FF0000`); nein: schwarz (`0x00000000`)
- Offset der ersten LED: `0 (Base)`

```
# Adressen nur, wenn der Export sie bestaetigt
li s1, SWITCHES_0_BASE
li s2, LED_MATRIX_0_BASE
Loop:
  lw t0, 0(s1)
  andi t1, t0, 8        # Maske Bit 3
  beq t1, zero, Off
  li t2, 0x00FF0000     # Rot
  j Store
Off:
  li t2, 0
Store:
  sw t2, 0(s2)
  j Loop
```

Notizen

beispiele/vl8_sw3_rot.s maskiert mit 8. 0b0000 und 0b0101 schreiben Schwarz. 0b1000 und 0b1101 schreiben Rot 0x00FF0000. Andere Schalter stören diese Zelle nicht. Das ist eine Pegelanzeige, keine Entprellung. s1 und s2 halten Adressen, t0 bis t2 die aktuellen Werte. Die Endlosschleife ist beabsichtigt. Ripes wurde nicht ausgeführt.
Kontrollfrage: Welches Farbwort schreibt 0b0101? Null. Das Lauflicht nutzt mehrere Wörter.

Folie 76 / 84

Labor-Aufgabe 2 (Ripes)

- Umlaufendes Lauflicht: ein grüner Pixel wandert und der vorige wird gelöscht
- Offset-Pointer startet bei 0; Delay-Loop für Sichtbarkeit
- Pointer += 4 für nächstes LED-Wort; am Ende zurücksetzen

```
# Lauflicht im I/O-Tab beobachten
```

Notizen

beispiele/vl8_lauflich.s setzt eine Zeile mit acht grünen Schritten. Der Offset läuft 0, 4 bis 28. Die Bytegrenze ist 32 und wird nicht beschrieben. Der alte Pixel wird gelöscht. Der Zähler 10000 wird je Pixel neu geladen und belegt keine Millisekunden. Ein Hin-und-her-Muster wäre ein anderes Programm.

Kontrollfrage: Bleibt der vorige Pixel an? Nein, er wird genullt. Danach ersetzt ein Dienstvertrag den Pixel.

Folie 77 / 84

Wechsel zu Venus: Konsolendienste

- Ripes schließen, Venus öffnen; Konsole unten
- Venus fängt `ecall` ab, wertet Service-ID in `a0` aus, schreibt in die Konsole
- Keine harten MMIO-Adressen — Arbeit an das OS delegieren

Notizen

Geöffnet wird die Kursvariante von Venus. Assembliert und ausgeführt wird dort. a0 ist die Dienstnummer, a1 das Argument. Die Konsole ist die Wirkung. Das ist kein nachgewiesener Kernel. Ripes darf offen bleiben. Ein Screenshot liegt nicht vor.
Kontrollfrage: Steht die Nummer in a7? In dieser Venus-Konvention nein. Die drei Dienste folgen.

Folie 78 / 84

Die Venus Syscall-Tabelle

- Essenzielle Services für das Labor (ID immer in `a0`):

`a0` (ID)	Name	Argumente	Resultat
1	Print Integer	`a1` = Zahl	Zahl auf Konsole
4	Print String	`a1` = Adresse	Text auf Konsole
10	Exit	keine	Programm endet

```
Hinweis: Venus hat keinen interaktiven Read-Int (anders als RARS).
```

Notizen

Dienst 1 druckt eine ganze Zahl aus a1. Dienst 4 druckt einen nullterminierten String, dessen Adresse in a1 steht. Dienst 10 beendet ohne Rückkehr. Ein Read-Int ist in dieser Standardtabelle nicht enthalten. Registererhaltung ist ein eigener Vertrag und hier nicht als universelle Venus-Garantie belegt.
Kontrollfrage: Ist a1 bei Dienst 4 der Text selbst? Nein, die Adresse. Der String wird angelegt.

Folie 79 / 84

Labor-Aufgabe 3 (Venus)

- Hello World: `.data` mit `.string "Hello World!\n"`
- `la a1, …` — Startadresse; `li a0, 4; ecall`
- Sauber beenden: `li a0, 10 / ecall`

```
la a1, msg
li a0, 4
ecall
li a0, 10
ecall
```

Notizen

beispiele/vl8_hello.s legt msg mit .asciiz an. .data und .text trennen Daten und Code. la lädt die Adresse, nicht das erste Zeichen. Das Nullbyte beendet den String. Die Ausgabe ist Hello World mit Zeilenumbruch, danach Exit. Eine konkrete Datenadresse wird nicht erfunden. .globl main passt nur, wenn der Simulator main startet.
Kontrollfrage: Druckt das Nullbyte? Nein. Als Nächstes ein Zahlenwert aus dem Speicher.

Folie 80 / 84

Labor-Aufgabe 4 (Venus)

- EVA ohne Tastatur: Wert in `.data` (z. B. `n: .word 21`)
- Laden, verdoppeln (Shift oder `add`), per Service 1 drucken
- Optional: Prompt-String per Service 4 vorher ausgeben
- Mit Service 10 beenden

```
lw t0, n
slli a1, t0, 1   # *2
li a0, 1
ecall
```

Notizen

EVA bedeutet Eingabe, Verarbeitung, Ausgabe. n ist das Wort 21. la und lw 0(Register) laden es. slli um 1 ergibt 42. Dienst 1 druckt 42, Dienst 4 den Zeilenumbruch, Dienst 10 beendet. Das Wort steht vor dem String, damit die Ausrichtung erhalten bleibt. lw t0, n ist keine garantierte Basisinstruktion. Es gibt keine interaktive Eingabe. Das Programm ist beispiele/vl8_eva.s und wurde nicht ausgeführt.
Kontrollfrage: Warum nicht zuerst der String? Damit das Wort ausgerichtet bleibt. Fehler liegen oft im Vertrag.

Folie 81 / 84

Debugging I/O (Best Practices)

- Ripes LED schwarz?
  - Gerät, Export, effektive Adresse, Maske und Farbwort prüfen
  - Ein Speicherbild allein beweist nicht die sichtbare Diodenwirkung
- Venus: Dienstnummer in `a0`, Argument in `a1`

```
Simulator als Spion: Register vor ecall prüfen.
```

Notizen

Bei Ripes werden Gerät, Export, effektive Adresse, Breite, Alignment, Maske, Farbwort und Store geprüft. Die Memory-Ansicht allein beweist weder RAM noch die Diodenwirkung. Bei Venus werden a0, a1, Wert gegen Zeiger, Terminator und Exit geprüft. Ein Refresh ist keine belegte allgemeine Heilung.
Kontrollfrage: Liegt eine falsche LED-Adresse immer im Heap? Nein. Der Registererhalt über ecall folgt.

Folie 82 / 84

Clobbering bei ecall (Warnung)

- `ecall` = ABI-Funktionsaufruf in den Kernel
- Der Clobber-Vertrag folgt aus der konkreten Umgebung, nicht aus der Funktions-ABI
- Venus-Dienste dieses Kurses: Rückgabe und erhaltene Register in der Venus-Tabelle nachlesen
- Nicht jeder `ecall` darf alle Caller-Saved-Register zerstören
- Venus erhält `t`-Register faktisch oft — das ist nicht garantiert und nicht portabel
- Schleifenzähler in `t0` + Print in der Schleife: auf anderen Systemen riskant
- Lösung: vor `ecall` auf Stack sichern oder `s0`–`s11` nutzen

Notizen

Der Clobber-Vertrag kommt aus der konkreten Umgebung. ecall ist kein normaler Funktionsaufruf. Daraus folgt nicht, dass alle caller-saved Register zerstört werden. RARS dokumentiert Erhaltung außer Ergebnisregistern. Für die gewählte Venus-Variante ist die Erhaltung ohne Prüfung offen. Ein s-Register ist nur sicher, wenn der Vertrag es sagt. Portabilität braucht außerdem passende Dienste.
Kontrollfrage: Ist s0 über jeden ecall portabel sicher? Nein. Die Zusammenfassung ordnet die Verträge.

Folie 83 / 84

Zusammenfassung Einheit 8

- MMIO: Hardware als RAM-Adresse — ohne Sonderbefehle
- Flüchtigkeit: MMIO ändert sich von selbst (Caching-Gefahr)
- Polling vs. Interrupts: Busy Waiting vs. Türklingel
- CSR-System: sichere Trap-Register (`mcause`, `mepc`, …)
- Syscalls (`ecall`): Brücke vom User-Mode zum Kernel

```
Von Transistoren bis zum Betriebssystem:
das Ökosystem der Computerarchitektur ist durchdrungen.
```

Notizen

MMIO ordnet Geräten Adressen zu. Masken isolieren Bits. RGB-Wörter kodieren Pixel. volatile und Cacheattribute sind verschiedene Ebenen. Polling und Interrupts organisieren Reaktionen. CSRs halten den Trap-Zustand. Syscalls folgen dem Vertrag der Umgebung. Das ist keine Beherrschung der gesamten Systemarchitektur. Der direkte MMIO-Pfad und der Dienstpfad bleiben getrennt.
Kontrollfrage: Ist Venus-Ausgabe ein Beweis für mtvec? Nein. Die Aufgaben prüfen das selbst.

Folie 84 / 84

Q&A und Hands-On

- Ripes oder Venus je nach Aufgabe öffnen
- Syscall-Tabelle als Referenz
- Arrays: Pointer +4 pro Wort; Strings: Null-Byte beachten
- Viel Erfolg beim Bau der Brücke zur Außenwelt



Notizen

Q&A bedeutet Questions and Answers. Ein 32-Bit-Wortarray schreitet um 4 Byte. Ein String ist eine Bytefolge mit Nullterminator, nicht generell plus 4. Ripes erwartet bei SW3 Rot und sonst Schwarz, das Lauflicht genau einen grünen Pixel. Venus erwartet Hello World und danach 42. 0b1101 und Maske 8 ergeben 8 und dann Rot. Druck und Freigabe zählen einen gehaltenen Pegel einmal, filtern Prellen aber nicht. MIE gleich 0 sperrt nicht ecall. 0x0000000B ist M-ecall, 0x8000000B ein External Interrupt. Die nächste Einheit wird hier nicht erfunden.
Kontrollfrage: Was steht bei Venus Print Integer in a0 und a1? 1 und der Zahlenwert. Damit sind Adresse, Maske und Dienstvertrag unterscheidbar.